SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Dry Run Evaluation

Course Code:	CSA17	**Course Title:**	Artificial Intelligence
CO Assessed:	CO1 – Apply (BL3)	**Assessment:**	Assessment Tool 2 – Dry Run Evaluation
Weightage:	35% of CIA	**Duration / Marks:**	60 mins / 100 marks
CO1 Statement:	Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)		
Student Name:	S Sandhiya	**Reg. No.:**	192321015
Section / Batch:	Slot D	**Date:**	10-10-2026

Code of Conduct

I S Sandhiya(192321015) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: S Sandhiya

Instructions

- Read each question carefully; answer all four questions (25 marks each, 100 in total).
- Show every dry-run step: the initial queue, stack, priority queue or belief state, each iteration, your assumptions and a clear final answer, and finish with the interpretation asked for.
- Follow the conventions stated in each question (alphabetical expansion, explored set, goal test on removal from the frontier, tie-breaking) and present tables neatly.
- Marks follow the Dry Run Evaluation rubric: understanding of the search problem, state expansion and dry run execution, application of the search algorithm, accuracy of the solution path, and presentation and logical reasoning.
- Upload in GitHub and GCR (question paper with Reg. No., Name, answers, rubrics and evaluation page at the end). Duration: 1 hour (15 minutes per question).

Section / Topic	Focus	Question Nos.
1. Foundations and History of AI	Four approaches to AI, Turing Test, eras and milestones	**Q2**
2. Intelligent Agents: Agents and Environments	Agent, percepts, actions, agent function and program, PEAS	**Q1**
3. The Concept of Rationality	Performance measure, percept sequence, rational vs omniscient	**Q1**
4. The Nature of Environments	Task-environment properties and their classification	**Q1**
5. The Structure of Agents	Simple reflex and model-based reflex agent programs	**Q1**
6. Problem-Solving Agents	Goal formulation, problem formulation, search, execution	**Q2**
7. Problem Formulation	States, initial state, actions, goal test, path cost; belief states	**Q2, Q4**
8. Searching for Solutions	Search graph, frontier, explored set, node expansion	**Q3**
9. Uninformed Search Strategies	Dry runs of BFS, DFS and uniform-cost search	**Q3**
10. Search with Partial Information	Sensorless problems, belief states, percept update	**Q4**

Annexure A – Dry Run Evaluation

Question 1: Agents, Environments and Rationality – Dry Run of Agent Programs

Topic: Intelligent agents, rationality, nature of environments, structure of agents | 15 minutes

A vacuum-cleaner agent lives in a world with two squares, A and B. At each time step it perceives [its location, status of its own square] and performs one action: Suck, Right, Left or NoOp. The agent starts in square A, both squares are initially Dirty, and dirt never reappears.

Performance measure over T = 6 time steps: +10 for every square cleaned (a Suck on a Dirty square) and −1 for every Left or Right move.

Agent X (simple reflex): if the current square is Dirty, Suck; otherwise move to the other square.

Agent Y (model-based reflex): keeps an internal state of the squares it knows to be clean. If the current square is Dirty, Suck; otherwise, if both squares are known clean, NoOp; otherwise move to the other square.

(a) Write the PEAS description (Performance measure, Environment, Actuators, Sensors) of this agent. **[4 Marks]**

(b) Classify the task environment on observability, number of agents, determinism, episodic/sequential, static/dynamic and discrete/continuous, with a one-phrase justification each. **[4 Marks]**

(c) Dry-run both agents for all 6 time steps. Complete the table (percept, action, and Y's internal state after the step). **[9 Marks]**

Step	X: Percept	X: Action	Y: Percept	Y: Action	Y: Known-Clean Squares
1					
2					
3					
4					
5					
6					

(d) Compute the total performance of X and of Y over the 6 steps. **[3 Marks]**

(e) Interpret: which agent is rational? Both agents receive the identical percept [B, Clean] at step 4 – explain why they act differently, using the structure of the two agent programs, and distinguish a rational agent from an omniscient agent. **[5 Marks]**

(a) PEAS Description (4 Marks)

PEAS stands for Performance measure, Environment, Actuators, Sensors.

Component	Description
P – Performance measure	+10 for each square cleaned; −1 for every Left or Right move.
E – Environment	Two squares, A and B, initially dirty.
A – Actuators	Suck, Right, Left, NoOp.
S – Sensors	Perceives the current location and the status (Dirty/Clean) of the current square.

(b) Classification of Task Environment (4 Marks)

Property	Classification	Justification
Observability	Partially observable	The agent sees only its current square's status.
Number of agents	Single-agent	Only one vacuum-cleaner agent operates.
Determinism	Deterministic	Each action has a predictable result.
Episodic/Sequential	Sequential	Each action affects the next state.
Static/Dynamic	Static	Dirt does not reappear, and the environment does not change independently.
Discrete/Continuous	Discrete	Locations, actions and dirt status are finite.

(c) Dry Run of Both Agents (9 Marks)

Initial condition: A = Dirty, B = Dirty. Both agents start at A.

Assumption: Each step's percept is received before the action, and the action changes the state for the next step. Agent Y initially knows neither square is clean.

Step	X: Percept	X: Action	Y: Percept	Y: Action	Y: Known-clean squares after step
1	[A, Dirty]	Suck	[A, Dirty]	Suck	
2	[A, Clean]	Right	[A, Clean]	Right	
3	[B, Dirty]	Suck	[B, Dirty]	Suck	
4	[B, Clean]	Left	[B, Clean]	NoOp	
5	[A, Clean]	Right	[B, Clean]	NoOp	
6	[B, Clean]	Left	[B, Clean]	NoOp	

Important note: Agent Y's internal state records the squares it knows to be clean. Once both A and B are known clean, Y chooses NoOp. Therefore, Y stays at B after step 3.

(d) Total Performance of X and Y (3 Marks)

Performance formula:

$$\text{Total score}=10(\text{dirty squares cleaned})-1(\text{moves})$$

Agent X:

- Squares cleaned: 2 → $2 \times 10 = 20$
- Moves: 4 (Right, Left, Right, Left) → -4

$$\boxed{X=20-4=16}$$

Agent Y:

- Squares cleaned: 2 → $2 \times 10 = 20$
- Moves: 1 (Right) → -1

$$\boxed{Y=20-1=19}$$

Therefore, Agent Y performs better than Agent X over the six time steps.

(e) Interpretation and Rationality (5 Marks)

1. Which agent is rational?

Agent Y is more rational in this scenario because it remembers which squares are clean and avoids unnecessary movements. Agent X follows a simple rule and continues moving even after both squares are clean.

2. Why do they act differently at step 4 despite receiving the same percept?

Both receive the percept [B, Clean], but their programs use different information.

- Agent X — Simple reflex: Uses only the current percept. Since B is clean, it follows the rule "otherwise move to the other square" and chooses Left.
- Agent Y — Model-based reflex: Uses the current percept and its internal state. Since it knows both A and B are clean, it chooses NoOp.

3. Rational agent vs. omniscient agent

Rational agent	**Omniscient agent**
Chooses the best action based on its percepts, available knowledge and expected performance.	Knows the actual state and the outcome of every possible action in advance.
May make a non-optimal decision because its information is limited.	Has complete knowledge of what will happen.

Question 2: Foundations and History of AI; Problem-Solving Agent and Problem Formulation

Topic: Foundations and history of AI, problem-solving agents, problem formulation | 15 minutes

Part A. Complete the table by classifying each system or idea under one of the four approaches to AI (thinking humanly, acting humanly, thinking rationally, acting rationally) and naming the era or milestone of AI history to which it belongs.

(a) Complete the table. **[8 Marks]**

System / Idea	**Approach to AI**	**Era / Milestone**
Turing Test proposed (1950)		
General Problem Solver (Newell and Simon, 1957)		
Aristotle's syllogisms and formal logic		
An agent that does the right thing to maximise expected performance		

Part B. A problem-solving agent has two jugs: X of capacity 4 litres and Y of capacity 3 litres, both initially empty and without measuring marks. It may fill a jug from a tap, empty a jug on the ground, or pour one jug into the other until the source is empty or the target is full. The goal is to have exactly 2 litres in jug X.

(b) Formulate the problem: state representation, initial state, actions, transition model, goal test and path cost. **[6 Marks]**

(c) Generate all successor states of (0, 0) and of (4, 0) (leave out actions that leave the state unchanged). **[3 Marks]**

(d) How many states does the state space contain? Write a shortest solution as a sequence of states and give its path cost. **[3 Marks]**

(e) Interpret: describe the four phases (goal formulation, problem formulation, search, execution) of a problem-solving agent for this task, and explain why the agent can execute the found solution without looking at new percepts. **[5 Marks]**

Part A — Four Approaches to AI (8 Marks)

System / Idea	**Approach to AI**	**Era / Milestone**
Turing Test proposed (1950)	Acting humanly	1950 — Turing Test and early AI foundations
General Problem Solver (Newell and Simon, 1957)	Thinking humanly	1950s — Early symbolic AI and cognitive simulation
Aristotle's syllogisms and formal logic	Thinking rationally	Ancient Greece — Foundations of formal logic
An agent that maximises expected performance	Acting rationally	Modern AI — Rational-agent approach

Easy way to remember:

- Thinking humanly = think like humans.
- Acting humanly = behave like humans.
- Thinking rationally = reason using logic.
- Acting rationally = choose the best action to achieve a goal.

Part B — Water Jug Problem Formulation (6 Marks)

Given:

- Jug X capacity = 4 litres.
- Jug Y capacity = 3 litres.
- Initial state = both jugs empty.
- Goal = exactly 2 litres in jug X.

1. State representation

Represent each state as:

$$ (X,Y) $$

Here, X is the amount of water in the 4-litre jug, and Y is the amount in the 3-litre jug.

Example: $(4,2)$ means X contains 4 litres and Y contains 2 litres.

2. Initial state

$$ \boxed{(0,0)} $$

3. Actions

- Fill jug X.
- Fill jug Y.
- Empty jug X.
- Empty jug Y.
- Pour X into Y.
- Pour Y into X.

4. Transition model

The transition model describes the resulting state after each action.

Examples:

- Fill X: $(x,y)\rightarrow(4,y)$
- Fill Y: $(x,y)\rightarrow(x,3)$
- Empty X: $(x,y)\rightarrow(0,y)$
- Empty Y: $(x,y)\rightarrow(x,0)$
- Pour X into Y until X is empty or Y is full.
- Pour Y into X until Y is empty or X is full.

5. Goal test

Check whether jug X contains exactly 2 litres:

$$ \boxed{X=2} $$

6. Path cost

Assume every action costs 1. The path cost is the total number of actions used to reach the goal.

$$ \boxed{\text{Path cost}=\text{Number of actions}} $$

Part C — Generate Successor States (3 Marks)

A successor state is the state obtained after performing a valid action.

1. Successors of (0, 0)

Both jugs are empty.

Action	Successor state
Fill X	(4, 0)
Fill Y	(0, 3)

Emptying or pouring leaves the state unchanged, so those actions are excluded.

Answer: $\boxed{(4,0), (0,3)}$

2. Successors of (4, 0)

Jug X is full, and jug Y is empty.

Action	Successor state
Fill X	No change
Fill Y	(4, 3)
Empty X	(0, 0)
Empty Y	No change
Pour X into Y	(1, 3)
Pour Y into X	No change

Answer: $\boxed{(4,3), (0,0), (1,3)}$

Part D — State Space and Shortest Solution (3 Marks)

1. How many states does the state space contain?

Jug X can hold 0, 1, 2, 3 or 4 litres: 5 possibilities.

Jug Y can hold 0, 1, 2 or 3 litres: 4 possibilities.

Therefore, the total number of possible states is:

$$ 5\times4=\boxed{20\text{ states}} $$

Of these, 14 states are reachable from $(0,0)$ using the allowed actions.

2. Shortest solution

The goal is to obtain exactly 2 litres in jug X.

0, 0

0, 3

3, 0

Start → Fill Y → Pour Y into X

3, 3

4, 2

0, 2

Fill Y → Pour Y into X until X is full → Empty X

(2, 0)

Goal reached

Pour Y into X

The complete state sequence is:

Path cost: There are 6 actions, each costing 1.

Part E — Four Phases of a Problem-Solving Agent (5 Marks)

Phase	**Explanation for the water jug problem**
1. Goal formulation	Decide to obtain exactly 2 litres in jug X.
2. Problem formulation	Define states, initial state \((0,0)\), valid actions, transition model, goal test and path cost.
3. Search	Explore possible states and find the shortest sequence that reaches \((2,0)\).
4. Execution	Perform the six actions in the planned order to reach the goal.

Why can the agent execute the solution without looking at new percepts?

The agent can execute the plan without checking new percepts because:

- The environment is assumed to be fully observable.
- The actions have predictable outcomes (deterministic environment).
- The capacities and initial state are known.
- No unexpected changes occur while executing the plan.

Question 3: Searching for Solutions – Uninformed Search Strategies

Topic: Searching for solutions, BFS, DFS, uniform-cost search | 15 minutes

Consider the weighted undirected graph below. The start node is S and the goal node is G. Use graph search with an explored set; expand neighbours alphabetically, test the goal when a node leaves the frontier, and break ties alphabetically. BFS and DFS ignore edge costs while searching, but path costs are measured with them.

Edge	**Cost**	**Edge**	**Cost**
S – A	1	B – D	1
S – B	4	C – G	6
A – C	2	D – G	2
A – D	5		

(a) Perform BFS from S (queue initially [S]). Do not add a node that is already explored or already in the queue. **[6 Marks]**

Iteration	Node Dequeued	Queue after Expansion	Explored Set
1			
2			
3			
4			
5			
6			

(b) Perform DFS from S using a stack (initially [S]); push children so that the alphabetically first child is popped first. Write the stack with its top on the left. **[5 Marks]**

Iteration	Node Popped	Stack after Pushing Children
1		
2		
3		
4		

(c) Perform uniform-cost search from S. Write queue entries as Node:g(parent) and replace an entry whenever a cheaper path to a frontier node is found. **[7 Marks]**

Iteration	Node Expanded	g(n)	Priority Queue after Expansion
1			
2			
3			
4			
5			
6			

(d) Complete the comparison table. **[4 Marks]**

Strategy	Path Found	Path Cost	Nodes Removed from Frontier	Optimal Here?
BFS				
DFS				
UCS				

(e) Interpret: explain why UCS removed as many nodes as BFS yet found a cheaper path, why DFS needed the fewest nodes here but cannot be trusted in general, and why an explored set is needed in graph search (use iteration 3 of BFS as an example). **[3 Marks]**

Given graph

Edge	Cost	Edge	Cost
S–A	1	B–D	1
S–B	4	C–G	6
A–C	2	D–G	2
A–D	5		

Start node: S. Goal node: G.

(a) Breadth-First Search (BFS) — 6 Marks

BFS uses a FIFO queue: First In, First Out. Expand neighbours alphabetically and do not add nodes already explored or already in the queue. Test the goal when it is dequeued.

Iteration	Node dequeued	Queue after expansion	Explored set
1	S	[A, B]	
2	A	[B, C, D]	
3	B	[C, D]	
4	C	[D, G]	
5	D	[G]	
6	G	[]	

(b) Depth-First Search (DFS) — 5 Marks

DFS uses a LIFO stack: Last In, First Out. The stack top is on the left. Push children in reverse alphabetical order so the alphabetically first child is popped first.

Iteration	Node popped	Stack after pushing children
1	S	[A, B]
2	A	[C, D, B]

3	C	[G, D, B]
4	G	[D, B]

The goal is tested when G is popped, so the search stops at iteration 4 without expanding G.

(c) Uniform-Cost Search (UCS) — 7 Marks

UCS expands the frontier node with the lowest cumulative path cost \(g(n)\). If a cheaper path to a frontier node is discovered, replace its existing entry.

Queue entries are written as Node:g(parent), with the lowest-cost entry first. For example, A:1(S) means node A has path cost 1 and parent S.

Iteration	**Node expanded**	**\(g(n)\)**	**Priority queue after expansion**
1	S	0	[A:1(S), B:4(S)]
2	A	1	[C:3(A), B:4(S), D:6(A)]
3	C	3	[B:4(S), D:6(A), G:9(C)]
4	B	4	[D:5(B), G:9(C)]
5	D	5	[G:7(D)]
6	G	7	[]

Important updates:

- At iteration 4, B provides a cheaper route to D: cost 5 instead of 6.
- At iteration 5, D provides a cheaper route to G: cost 7 instead of 9.
- At iteration 6, G is removed from the frontier and the search stops.

(d) Comparison Table — 4 Marks

Strategy	**Path found**	**Path cost**	**Nodes removed from frontier**	**Optimal here?**
BFS	S → A → C → G	9	6	No
DFS	S → A → C → G	9	4	No
UCS	S → B → D → G	7	6	Yes

Nodes removed include the goal node, because the goal is tested when it leaves the frontier.

(e) Interpretation — 3 Marks

1. Why did UCS remove as many nodes as BFS but find a cheaper path?

Both BFS and UCS removed 6 nodes. However, BFS searches by depth and ignores edge costs, whereas UCS considers the cumulative path cost. Therefore, UCS found the cheaper path with cost 7, compared with BFS's cost of 9.

2. Why did DFS need fewer nodes, and why can't it be trusted in general?

DFS reached G after removing only 4 nodes because it followed the S–A–C–G branch first. However, DFS depends on the order of exploration and can find a longer path or spend a long time exploring an unhelpful branch. It does not guarantee the lowest-cost solution.

3. Why is an explored set needed?

An explored set prevents revisiting already expanded nodes and helps avoid repeated work or infinite loops in graphs containing cycles.

For example, at BFS iteration 3, node B has neighbour S. Since S is already in the explored set, BFS does not add S back to the queue. Also, D is already in the queue, so it is not added again.

Question 4: Search with Partial Information – Belief States

Topic: Problem formulation, search with partial information (sensorless and percept-based belief-state search) | 15 minutes

A vacuum agent lives in a two-square world (Left square, Right square), each square being Clean (C) or Dirty (D). Actions: Left, Right and Suck (Left in the left square and Right in the right square leave the agent where it is; Suck cleans the current square). Physical states are written (Agent, Left square, Right square):

State	Description	State	Description
S1	(L, D, D)	S5	(L, C, D)
S2	(R, D, D)	S6	(R, C, D)
S3	(L, D, C)	S7	(L, C, C)
S4	(R, D, C)	S8	(R, C, C)

(a) Initially the agent has no sensors and does not know its location or the dirt status. Formulate the problem over belief states: initial belief state, actions, goal test and path cost. **[3 Marks]**

(b) Apply the action sequence [Right, Suck, Left, Suck] and write the belief state after each action. **[6 Marks]**

Step	Action	Belief State after Action
0	(start)	{S1, S2, S3, S4, S5, S6, S7, S8}
1	Right	
2	Suck	
3	Left	
4	Suck	

(c) Now the agent has a local sensor that reports [location, status of the current square]. Starting again from the initial belief state, the first percept is [Left, Dirty]; the agent then performs Suck and

perceives [Left, Clean]; it then performs Right and perceives [Right, Dirty]. Fill in the belief state after each action (prediction) and after each percept (update). **[8 Marks]**

Step	Action	After Action (Predicted)	Percept	After Percept (Updated)
0	(start)	{S1, S2, S3, S4, S5, S6, S7, S8}	[Left, Dirty]	
1	Suck		[Left, Clean]	
2	Right		[Right, Dirty]	

(d) How many different belief states are possible when there are 8 physical states? Why does this make the problem harder than searching over physical states? **[3 Marks]**

(e) Interpret: explain why the percepts in part (c) shrink the belief state, how this differs from the sensorless case in part (b), and why the sensorless plan can still be called rational. **[5 Marks]**

(a) Formulate the Problem over Belief States (3 Marks)

A belief state is a set of physical states in which the agent could possibly be.

1. Initial belief state

The agent does not know its location or the dirt status of either square.

2. Actions: Left, Right, Suck.

3. Goal test: Every physical state in the belief state must have both squares clean (C). The goal belief state is a subset of \(\{S7,S8\}\).

4. Path cost: Assume each action costs 1. Total path cost is the number of actions performed.

(b) Belief State after Each Action (6 Marks)

In the sensorless case, the agent cannot observe its location or dirt status. After each action, it predicts all possible resulting states.

Step	Action	Belief state after action
0	Start	
1	Right	
2	Suck	
3	Left	
4	Suck	

How to understand the result:

- Right: The agent moves right if it is in the left square; otherwise, it stays right. All eight possibilities remain possible.

- Suck: The square where the agent is located becomes clean. The resulting states are S2, S4, S6 and S8.
- Left: The agent moves to the left square. The possible states become S1, S3, S5 and S7.
- Suck: The left square becomes clean, leaving S5 and S7 possible.

The final belief state is \(\{S5,S7\}\). Both states have the left square clean, but the right square is dirty in S5. Therefore, the sequence has not guaranteed that both squares are clean.

(c) Belief-State Search with Percepts (8 Marks)

Here, the agent has a sensor. It first predicts the possible states after an action and then updates the belief state using the new percept.

Step	Action	After action (Predicted)	Percept	After percept (Updated)
0	Start		[Left, Dirty]	
1	Suck		[Left, Clean]	
2	Right		[Right, Dirty]	

Step-by-step explanation

Step 0 — Percept [Left, Dirty]

The agent is in the left square, and the left square is dirty. Only S1 and S3 match this information.

\[B=\boxed{\{S1,S3\}} \]

Step 1 — Action Suck

- S1 = (L, D, D) becomes S5 = (L, C, D).
- S3 = (L, D, C) becomes S7 = (L, C, C).

Predicted belief state:

\[\{S5,S7\} \]

The percept [Left, Clean] matches both states, so the updated belief state remains \(\{S5,S7\}\).

Step 2 — Action Right

- S5 = (L, C, D) becomes S6 = (R, C, D).
- S7 = (L, C, C) becomes S8 = (R, C, C).

Predicted belief state:

\[\{S6,S8\} \]

The percept [Right, Dirty] means the right square is dirty. Only S6 matches this percept.

\[B=\boxed{\{S6\}} \]

(d) Number of Possible Belief States (3 Marks)

There are 8 physical states. Every physical state can either be included in or excluded from a belief state.

Therefore,

$$ \text{Number of belief states}=2^8 $$

$$ \boxed{256\text{ belief states}} $$

This count includes the empty set. If only nonempty belief states are counted, there are $256-1=255$.

Why is belief-state search harder?

The agent searches over sets of physical states rather than individual physical states. The number of possible belief states grows exponentially as the number of physical states increases.

(e) Interpretation (5 Marks)

1. Why do percepts shrink the belief state?

A percept provides new information and eliminates physical states that do not match it. For example, [Left, Dirty] reduces the initial belief state from 8 possibilities to just $\{S1,S3\}$.

2. How is this different from the sensorless case?

- Sensorless search: The agent cannot observe the world, so it tracks all possible states after each action.
- Percept-based search: The agent receives observations and removes states inconsistent with those observations.

3. Why can the sensorless plan still be called rational?

A rational agent chooses actions that maximise expected performance using the information available to it. Even without sensors, it can plan actions that work across multiple possible states.

However, the sequence in part (b) is not a guaranteed successful plan, because its final belief state $\{S5,S7\}$ still includes a state where the right square is dirty. A rational agent should choose a plan that guarantees the goal if such a plan exists and guaranteed success is the objective.

Quick revision:

- Physical states = 8.
- Possible belief states = 256, including the empty set.
- Sensorless agent = predicts possible states.
- Agent with sensors = predicts, then updates using percepts.
- Goal belief state = every possible state has both squares clean.

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Search Problem	**20**	Clearly understands the search problem, initial state, goal state, and search strategy.	Understands the problem with minor misunderstandings.	Demonstrates partial understanding of the search problem.	Shows little or no understanding of the search problem.
State Expansion and Dry Run Execution	**30**	Correctly traces every iteration, expands nodes accurately, and maintains the Queue/Stack/Priority Queue without errors.	Performs most iterations correctly with minor mistakes in state expansion or data structure updates.	Performs only some iterations correctly; several errors in tracing or node expansion.	Unable to correctly perform the dry run or expand states.
Application of Search Algorithm	**20**	Applies the selected algorithm (BFS/DFS/UCS) correctly, following all algorithmic rules.	Applies the algorithm correctly with a few procedural errors.	Applies only basic algorithm steps with noticeable mistakes.	Fails to apply the correct search algorithm.
Accuracy of Solution Path	**20**	Identifies the correct traversal order/solution path and goal state with proper justification.	Solution path is mostly correct with minor errors.	Partially correct solution path; goal may not be reached correctly.	Incorrect or incomplete solution path.
Accuracy of Solution Path	**10**	Queue/Stack/Priority Queue updates, state diagrams, and explanations are neat, organized, and logically presented.	Presentation is clear with minor organizational issues.	Limited explanation and organization; reasoning is partially clear.	Poor presentation with unclear or missing reasoning.

Q. No. / Criteria	Understanding of Search Problem	State Expansion and Dry Run Execution	Application of Search Algorithm	Accuracy of Solution Path	Accuracy of Solution Path	Sub. Total (20)	Total %
1							
2							
3							

4							
Total							

Faculty In-charge	**Course Coordinator**	**Program Director**
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____